EDA (IT462): Insem 2 Cheatsheet, Part I

Krushna Parmar 202611032 Every code line was run on `sns.load_dataset("titanic")`.

0. The exam, and where to look

Insem 2 syllabus (book Ch. 24): collection & sampling, import & cleaning, missing data, descriptive summary, univariate & bivariate analysis, outliers, transformation (scaling, function, power, binning, encoding). Insem 1 format: Part A NOIR labels, Part B true/false, Part C one-line Python. The model paper adds theory + code questions ("explain … and print … for titanic").

Model-paper question → section

skewness: types, detect, titanic fare	§5
column names as a list, 3 ways	§1
LOF vs DBSCAN vs Isolation Forest + code	§10
five-number summary for titanic	§5
MCAR / MAR / MNAR, one example, implications	§9
four normalization methods + code	§11
four function + two power transformers	§11
which plot for which data	§6
Part C one-liners (NumPy / pandas)	§2–4

How to answer a theory + code question: definition in one line → formula → types or cases → when to use / limitation → code → read the output in one line.

1. Setup, loading, first look

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from scipy import stats
df = sns.load_dataset("titanic")        # 891 x 15
# other sources
pd.read_csv("f.csv", na_values=["?", "-", "NA"])
pd.read_excel("f.xlsx", sheet_name="Sheet1")
pd.read_json("f.json"); pd.read_csv("https://.../f.csv")
pd.read_html(url)[0]                    # first HTML table
df.shape; df.info(); df.dtypes; df.head(); df.tail(3)
df.sample(5, random_state=42)           # random rows, fixed seed
df.describe(); df.describe(include="all")
```

Column names as a Python list (six ways)

```
list(df.columns)          df.columns.tolist()
df.columns.values.tolist()   [c for c in df]
list(df.keys())           [*df]
```

Numeric-only and categorical-only

```
num = df.select_dtypes(include="number")   # or np.number
cat = df.select_dtypes(exclude="number")   # text, category, bool
num_cols = num.columns.tolist()
df.select_dtypes(include=["object","category"])  # text in pandas 2
df.mean(numeric_only=True); df.corr(numeric_only=True)
df[num_cols].agg(["mean", "median", "std", "skew"])
```

Trap

`df.corr()` on a frame with text columns raises an error in pandas ≥ 2: pass `numeric_only=True` or select numbers first.

Variations

```
pd.read_csv(f, sep=";", header=0, usecols=["id","marks"],
            index_col="id", nrows=100, skiprows=[1],
            parse_dates=["date"], na_values="?")
df.dtypes.value_counts()          # how many columns of each type
df.memory_usage(deep=True).sum()  # bytes
[c for c in df if pd.api.types.is_numeric_dtype(df[c])]
df.select_dtypes(include=["int64","float64"]).columns.tolist()
len(df); len(df.columns); df.size    # rows, columns, cells
```

2. Insem 1 Part C, corrected

These were the one-liners asked; the marked mistakes are fixed here.

```
a = np.arange(1, 21)                  # 1..20; NOT np.arange([1,21])
np.mean(a), np.median(a), np.std(a)   # std(a, ddof=1) = sample SD
df.sort_values("Score", ascending=False)   # not .sort(reverse=True)
M[1]                                  # 2nd row of a 2-D array
(arr < 0).sum()                       # count negatives
pd.DataFrame({"a": [1, 2], "b": [3, 4]})   # dict -> DataFrame
df.sample(5, random_state=42)         # random rows, fixed seed
df.nunique()                          # unique count PER COLUMN
pd.concat([df1, df2], axis=0)         # stack rows (not hstack)
df.drop_duplicates()                  # keep="first" by default
df["new"] = df["c1"] - df["c2"]
df.groupby("Dept")["Salary"].mean()
df["category"].value_counts().head(3)      # top 3 frequent
df.corr(numeric_only=True)
df.to_csv("clean.csv", index=False)
```

NOIR answers that were marked wrong

race position	O	no. of pets	R
SAT score	I	year of purchase	I
no. of rooms	R	customer segment	N
satisfaction 1–10	O	disease grade 1–4	O
temperature °F, pH, clock time	I	Likert, star rating	O

Rules: categories with no order N; ordered labels or ranks O; equal steps but no true zero (dates, years, °C/°F, pH, SAT, IQ) I; true zero so ratios make sense (counts, money, lengths, time durations, kelvin) R.

True / false that come back

T: difference of interval data meaningful; mode for nominal; SD meaningful for ratio; bar chart for categorical; scatterplot for two numeric; frequency table for all scales; percentiles for ordinal; ratio supports division. **F**: mean best for ordinal (use median); boxplot shows three numbers (five); histogram for nominal (bar chart); correlation implies causation; zero in interval means absence; nominal has addition; ordinal lacks order.

What each scale allows

N	=, ≠; count, mode, % ; bar/pie; χ^2, Cramér's V
O	+ order <, >; median, percentiles, IQR; Spearman, Kendall
I	+ differences; mean, SD, z-scores, Pearson; no ratios
R	+ ratios ("twice"), CV, geometric mean, log transform

3. NumPy essentials

```
A = np.arange(1, 26).reshape(5, 5)
A.ndim, A.shape, A.size, A.dtype      # 2, (5,5), 25, int64
A[0], A[-1], A[:, 0], A[:, -1]        # first/last row, col
A[1:4, 1:4]                           # middle 3x3
A[A % 2 == 0]; A[A > 15]              # Boolean masks
A[::-1]; A[:, ::-1]                   # reverse rows / columns
np.linspace(0, 1, 5); np.zeros((2,3)); np.ones(4); np.eye(3)
A.astype("float64"); A.reshape(-1)    # -1: infer the size
np.clip(marks + 5, None, 100)         # bonus capped at 100
np.where(marks >= 50, "pass", "fail")
np.select([m >= 90, m >= 50], ["A", "B"], default="F")
np.percentile(m, [25, 50, 75]); np.var(m, ddof=1)
A + np.array([1, 2, 3, 4, 5])         # broadcast (5,5)+(5,)
np.unique(x, return_counts=True)
```

Broadcasting: shapes are compared from the right; each pair must be equal or one of them 1. `(3,3)+(3,)` works; `(3,3)+(2,)` errors.

4. pandas essentials

```
df.loc[0, "age"]; df.iloc[0, 3]              # label vs position
df.loc[df["age"] > 60, ["age", "fare"]]
df.query("age > 60 and sex == 'male'")
df[df["embarked"].isin(["C", "Q"])]
df.nlargest(3, "fare")
df.sort_values(["pclass","fare"], ascending=[True,False])
df["fare"].rank(ascending=False)
df.rename(columns={"sex": "gender"}); df.drop(columns=["deck"])
g = df.groupby("class", observed=True)["fare"]
g.agg(["mean", "median", "count"])    # one row per group
g.transform("mean")                   # same length as df
df.pivot_table(values="fare", index="class",
               columns="sex", aggfunc="mean", observed=True)
```

```
pd.crosstab(df["class"], df["survived"], normalize="index")
df["sex"].map({"male": 0, "female": 1})
df["fare"].apply(lambda x: round(x))
df["town"].replace({"Southampton": "S'ton"})
pd.merge(a, b, on="ID", how="left")   # inner/outer/left/right
pd.concat([a, b], axis=1)             # side by side
```

agg vs transform: agg reduces each group to one value; transform broadcasts it back to every row. **merge vs concat**: merge matches a key column (SQL join); concat just stacks.

Variations

```
df["fare"].idxmax(); df.loc[df["fare"].idxmax(), "who"]   # row of max
df["fare"].where(df["fare"] < 100)        # keep, else NaN
df["fare"].mask(df["fare"] > 100, 100)    # replace where True
df["age"].between(18, 60).sum()
df["fare"].cumsum(); df["fare"].pct_change(); df["fare"].shift(1)
df["fare"].rolling(5).mean()              # moving average
df.apply(lambda r: r["sibsp"] + r["parch"], axis=1)   # row-wise
df["embarked"].value_counts(normalize=True, dropna=False)
df.duplicated(keep=False).sum()           # all copies, not extras
df[["sex","age","fare"]].melt(id_vars="sex")          # wide -> long
df.groupby(["class","sex"], observed=True)["survived"].mean().unstack()
df.groupby("class", observed=True).agg(avg_fare=("fare","mean"),
        n=("fare","size"), max_age=("age","max"))     # named agg
```

Strings and dates

```
t = df["embark_town"].dropna()
t.str.upper(); t.str.len(); t.str.contains("ton"); t.str[:3]
t.str.replace("o", "0")
names.str.split(",").str[0]               # surname
names.str.extract(r",\s*(\w+)\.")         # title: Mr / Mrs
d = pd.to_datetime(s, errors="coerce")    # bad dates -> NaT
d.dt.year; d.dt.month_name(); d.dt.dayofweek; d.dt.to_period("M")
(d.max() - d.min()).days
```

5. Descriptive summary, skewness, five numbers

```
s = df["fare"]
s.mean(); s.median(); s.mode()[0]
s.var(); s.std()                  # pandas: ddof=1 (sample)
s.max() - s.min()                 # range
s.quantile(.75) - s.quantile(.25)     # IQR
```

Five-number summary

min, Q_1, median, Q_3, max: the centre, the spread of the middle half, and the extremes, in five numbers. It is exactly what a box plot draws. Titanic fare: 0, 7.91, 14.45, 31.00, 512.33.

```
s.quantile([0, .25, .5, .75, 1])
np.percentile(s, [0, 25, 50, 75, 100])
s.describe()[["min", "25%", "50%", "75%", "max"]]
# whole titanic (all numeric columns)
df.select_dtypes("number").describe() \
  .loc[["min", "25%", "50%", "75%", "max"]]
```

Skewness: what, types, how to detect

Skewness measures asymmetry: which tail is longer.

$$g_1 = \frac{\frac{1}{n}\sum(x_i - \bar{x})^3}{s^3}, \qquad \text{Pearson 2} = \frac{3(\bar{x} - \text{median})}{s}.$$

- **Positive (right)**: long right tail; mean > median > mode. Income, fare.
- **Negative (left)**: long left tail; mean < median < mode. Easy-exam marks, age at death.
- **Zero**: symmetric; mean ≈ median (normal, uniform).

Rule of thumb: $|g_1| < 0.5$ roughly symmetric; 0.5–1 moderate; > 1 highly skewed. Skew = 0 does not prove symmetry (book Ch. 11). **Statistically**: `.skew()`, `stats.skew`, compare mean and median, Pearson's coefficient. **Graphically**: histogram / KDE (long tail), box plot (median off-centre, longer whisker, outliers on one side), Q-Q plot (points bend away from the line).

```
s.skew()                          # 4.787  highly right-skewed
stats.skew(s, bias=False)         # same number
3 * (s.mean() - s.median()) / s.std()   # 1.07
s.kurt()                          # 33.4 excess: very heavy tails
df.select_dtypes("number").skew().sort_values()
fig, ax = plt.subplots(1, 3, figsize=(12, 3))
sns.histplot(s, kde=True, ax=ax[0])
ax[0].axvline(s.mean(), c="r"); ax[0].axvline(s.median(), c="g")
sns.boxplot(x=s, ax=ax[1])
stats.probplot(s, dist="norm", plot=ax[2])   # Q-Q plot
```

Fix right skew with log / sqrt / Box–Cox / Yeo–Johnson (§11): fare skew 4.79 → log1p 0.39 → Yeo–Johnson −0.04.

Variations

```
df.describe(percentiles=[.1, .9])
df.agg({"age": ["mean","median"], "fare": ["min","max"]})
df.groupby("class", observed=True)["age"].describe()
s.std() / s.mean()                # coefficient of variation
(s - s.mean()).abs().mean(); s.sem()    # mean abs deviation, std error
stats.trim_mean(s.dropna(), 0.1)  # 10% trimmed mean
stats.gmean(f[f > 0]); stats.hmean(f[f > 0])   # ratio data only
np.average(df["fare"], weights=df["pclass"])   # weighted mean
stats.zscore(s, nan_policy="omit")
stats.percentileofscore(s.dropna(), 30)        # percentile rank of 30
freq = pd.cut(s, bins=[0, 18, 40, 60, 80]).value_counts().sort_index()
pd.DataFrame({"freq": freq, "rel": freq/freq.sum(),
              "cum": freq.cumsum()})          # frequency table
stats.kurtosis(s.dropna())                    # excess (Fisher)
stats.kurtosis(s.dropna(), fisher=False)      # Pearson (normal = 3)
```

Titanic age: mean 29.70, median 28, mode 24, SD 14.53, CV 0.49, skew 0.39, five numbers 0.42, 20.12, 28, 38, 80. Kurtosis: > 0 leptokurtic (heavy tails), < 0 platykurtic, 0 mesokurtic.

6. Which plot for which data

Pick the plot from the data types

data	plots
1 numeric	histogram, KDE, box, violin, ECDF, Q-Q
1 categorical (N, O)	count/bar chart, pie (few levels)
numeric vs numeric	scatter, regplot, hexbin (many points), line (ordered x / time), joint
categorical vs numeric	side-by-side box, violin, strip, swarm, bar of means
categorical vs categorical	grouped / stacked bar, crosstab heatmap, mosaic
many numeric	pairplot, correlation heatmap, clustermap
missingness	msno bar, matrix, heatmap, dendrogram

```
# one numeric
sns.histplot(data=df, x="age", bins=30, kde=True)
sns.kdeplot(data=df, x="age", fill=True)
sns.boxplot(data=df, y="fare"); sns.violinplot(data=df, y="age")
sns.ecdfplot(data=df, x="age")
# one categorical
sns.countplot(data=df, x="class")
df["class"].value_counts().plot.bar()
df["embark_town"].value_counts().plot.pie(autopct="%1.1f%%")
# numeric vs numeric
sns.scatterplot(data=df, x="age", y="fare", hue="survived")
sns.regplot(data=df, x="age", y="fare", scatter_kws={"alpha": .3})
sns.jointplot(data=df, x="age", y="fare", kind="reg")
# categorical vs numeric
sns.boxplot(data=df, x="class", y="fare")
sns.violinplot(data=df, x="class", y="age", hue="sex", split=True)
sns.stripplot(data=df, x="class", y="age", jitter=True, alpha=.5)
sns.barplot(data=df, x="class", y="fare", errorbar="sd")
# categorical vs categorical
sns.countplot(data=df, x="class", hue="survived")
pd.crosstab(df["class"], df["survived"]).plot.bar(stacked=True)
sns.heatmap(pd.crosstab(df["class"], df["sex"]), annot=True, fmt="d")
# many numeric
sns.heatmap(df.corr(numeric_only=True), annot=True, fmt=".2f",
            cmap="coolwarm", vmin=-1, vmax=1)
sns.pairplot(df[["age","fare","pclass","survived"]].dropna(),
             hue="survived", diag_kind="kde")
```

Every numeric column at once

```
df.select_dtypes("number").hist(bins=20, figsize=(10, 6))
fig, axes = plt.subplots(2, 3, figsize=(12, 6))
for ax, col in zip(axes.ravel(), df.select_dtypes("number").columns):
    sns.boxplot(x=df[col], ax=ax); ax.set_title(col)
plt.tight_layout(); plt.show()
```

Box vs histogram: the box shows median, quartiles and outliers compactly and compares groups; the histogram shows shape (modes, gaps). **Violin**: box + KDE, shows bimodality. **Strip vs swarm**: strip jitters (points can overlap); swarm never overlaps (small n only). **Line vs scatter**: line only when x is ordered (time).

The same plots in matplotlib / pandas

```
plt.hist(f, bins=30, edgecolor="k")
plt.boxplot(df["age"].dropna(), vert=False)
plt.scatter(df["age"], f, alpha=.4, c=df["survived"], cmap="
    ↪ coolwarm")
vc = df["class"].value_counts()
plt.bar(vc.index.astype(str), vc.values)
plt.pie(vc, labels=vc.index, autopct="%1.1f%%", explode=[.1, 0, 0])
df["fare"].plot(kind="hist", bins=30, title="Fare")
df.plot(kind="scatter", x="age", y="fare")
df.boxplot(column="fare", by="class")
pd.plotting.scatter_matrix(df[["age","fare","pclass"]], figsize
    ↪ =(6,6))
fig, ax = plt.subplots()
ax.plot(x, y, "r--o", label="line")
ax.set(title="t", xlabel="x", ylabel="y"); ax.legend()
ax.annotate("peak", xy=(2,4), xytext=(2.5,4.2),
            arrowprops=dict(arrowstyle="->"))
fig.savefig("plot.png", dpi=150)
```

More seaborn

```
corr = df.corr(numeric_only=True)
mask = np.triu(np.ones_like(corr, dtype=bool))      # lower
    ↪ triangle
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", center=0)
sns.displot(data=df, x="age", hue="sex", kind="kde")
sns.catplot(data=df, x="class", y="fare", kind="box", col="sex")
g = sns.FacetGrid(df, col="survived"); g.map(sns.histplot, "age")
sns.pairplot(df[["age","fare","survived"]].dropna(), corner=True)
sns.lineplot(data=df, x="pclass", y="fare", errorbar="sd")
```

7. Bivariate statistics

Which coefficient

numeric–numeric, linear	Pearson r
monotone / ordinal / outliers	Spearman ρ, Kendall τ
binary–numeric	point-biserial
categorical–categorical	χ^2 test, Cramér's V

$\text{cov} = \frac{1}{n-1}\sum(x-\bar{x})(y-\bar{y})$, $r = \frac{\text{cov}}{s_x s_y} \in [-1, 1]$.

```
x, y = df["age"], df["fare"]
x.cov(y); x.corr(y)                          # Pearson (pairwise NaN drop)
x.corr(y, method="spearman"); x.corr(y, method="kendall")
m = df[["age","fare"]].dropna()
stats.pearsonr(m["age"], m["fare"])    # (r, p-value)
stats.pointbiserialr(df["survived"], df["fare"])
ct = pd.crosstab(df["sex"], df["survived"])
chi2, p, dof, expected = stats.chi2_contingency(ct)
stats.contingency.association(ct, method="cramer")
```

Variations

```
df.cov(numeric_only=True)                    # covariance matrix
num = df.select_dtypes("number")
pairs = [(a, b, *stats.pearsonr(num[[a,b]].dropna()[a],
                                num[[a,b]].dropna()[b]))
         for i, a in enumerate(num) for b in list(num)[i+1:]]
pd.DataFrame(pairs, columns=["x","y","r","p"]).sort_values("r")
df.groupby("survived")["fare"].mean()      # numeric by category
stats.ttest_ind(df.loc[df.survived==1, "fare"],
                df.loc[df.survived==0, "fare"], equal_var=False)
```

Titanic: fare–pclass Pearson -0.549, Spearman -0.688; age–fare 0.096; survived–fare point-biserial 0.257; sex vs survived $\chi^2 = 260.7$, $p \approx 10^{-58}$, Cramér's $V = 0.543$.

Sampling, in words

SRS: every subset equally likely. **Stratified**: split into groups, sample each (keeps proportions). **Cluster**: pick whole groups at random. **Systematic**: every k-th. **Convenience / voluntary**: biased. Sampling bias: the frame or the response differs from the population.

8. Distributions and sampling

```
rng = np.random.default_rng(42)
rng.normal(50, 10, 100); rng.uniform(5, 10, 100)
rng.binomial(10, .3, 100); rng.poisson(2, 100); rng.exponential(3,
    ↪ 100)
stats.norm.cdf(115, 100, 15)   # P(X<115) = 0.841
stats.norm.ppf(.975)           # 1.96
stats.binom.pmf(3, 10, .3)     # 0.267
# exponential restricted to [2, 10] (truncated)
stats.truncexpon(b=(10-2)/3, loc=2, scale=3).rvs(100, random_state
    ↪ =0)
df.sample(n=100, random_state=1)                          # SRS
df.groupby("class", observed=True, group_keys=False) \
  .sample(frac=.1, random_state=1)                        #
    ↪ stratified
df.iloc[::10]                                             #
    ↪ systematic
from sklearn.model_selection import train_test_split
tr, te = train_test_split(df, test_size=.2,
                          stratify=df["survived"], random_state=1)
```

Every `scipy.stats` distribution has `pdf`/`pmf`, `cdf`, `ppf` (inverse cdf), `rvs`.

9. Missing data

MCAR, MAR, MNAR on one example (model-paper question)

Survey of employees: **age** always recorded, **income** sometimes missing. R = missing indicator.

- **MCAR**: some forms were lost in the post at random. $P(R \mid Y_{obs}, Y_{mis}) = P(R)$. Missing rows look like everyone else.
- **MAR**: younger employees skip the income question more often; given age, it is random. $P(R \mid Y_{obs}, Y_{mis}) = P(R \mid Y_{obs})$.
- **MNAR**: high earners hide income *because* it is high. R depends on the missing value itself.

Implications. MCAR: deleting rows loses power but no bias; any method is valid. MAR: complete-case analysis is biased (too few young people); impute *using* the explaining column (group-wise, regression, KNN, MICE) or use maximum likelihood; ignorable. MNAR: every standard method is biased (mean income underestimated); needs a model of why data are missing, sensitivity analysis, or extra data; non-ignorable. MCAR can be tested (Little's test, χ^2, H_0: MCAR); MAR vs MNAR cannot be told apart from the data alone.

Detect

```
df.isnull().sum()                          # age 177, deck 688,
    ↪ embarked 2
df.isna().mean().mul(100).round(1)         # % missing per column
df.isnull().any(axis=1).sum()              # rows with any NaN
df.groupby(df["age"].isna())["pclass"].mean()  # missing vs observed
import missingno as msno
msno.bar(df); msno.matrix(df)
msno.heatmap(df); msno.dendrogram(df)
```

Treat

```
df.dropna()                                # complete case (listwise)
df.dropna(subset=["age"]); df.dropna(how="all")
df.dropna(axis=1, thresh=int(.7 * len(df)))   # keep cols >= 70%
    ↪ full
df["age"].fillna(df["age"].median())       # skewed numeric: median
df["embarked"].fillna(df["embarked"].mode()[0])   # categorical:
    ↪ mode
df["age"].fillna(df.groupby(["pclass","sex"])["age"]
                 .transform("median"))     # group-wise: suits MAR
df["age"].ffill(); df["age"].bfill(); df["age"].interpolate()  #
    ↪ ordered
df["age_missing"] = df["age"].isna().astype(int)   # keep a flag
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
nums = ["age", "fare", "sibsp", "parch"]
SimpleImputer(strategy="median").fit_transform(df[nums])
KNNImputer(n_neighbors=5).fit_transform(df[nums])
IterativeImputer(random_state=0).fit_transform(df[nums])  # MICE-
    ↪ like
```

Trap

Mean/median imputation shrinks the variance and puts a spike at one value; flag it. msno is visual only (non-confirmatory): it cannot prove MCAR or detect MNAR.

Variations

```
raw = raw.replace({-999: np.nan, "?": np.nan, "N/A": np.nan})  #
    ↪ placeholders
raw.fillna({"a": raw["a"].median(), "b": "unknown"})  # per column
df["age"].interpolate(method="linear")
df.dropna(thresh=13)                       # keep rows with >= 13 non-NaN
from sklearn.impute import MissingIndicator
MissingIndicator().fit_transform(df[["age","fare"]])
SimpleImputer(strategy="constant", fill_value=0).fit_transform(df[[
    ↪ "age"]])
SimpleImputer(strategy="mean", add_indicator=True).fit_transform(df
    ↪ [["age"]])
# is age missing at random w.r.t. other columns? (MCAR check)
miss = df["age"].isna()
```

```
stats.ttest_ind(df.loc[miss,"fare"], df.loc[~miss,"fare"],
      ↪ equal_var=False)
stats.chi2_contingency(pd.crosstab(miss, df["pclass"]))  # p = 1e
      ↪ -10
```

Titanic: age missing 13.9% in 1st class, 6.0% in 2nd, 27.7% in 3rd ($p \approx 10^{-10}$), so age is *not* MCAR; it depends on class: MAR at least. Impute it by class (and sex).

10. Outliers

Univariate rules (titanic fare)

z-score: $|z| = \frac{|x-\bar{x}|}{s} > 3$ (20 rows; assumes normal).
IQR (Tukey): outside $[Q_1 - 1.5\,\text{IQR},\ Q_3 + 1.5\,\text{IQR}] = [-26.7,\ 65.6]$ (116 rows).
Modified z: $0.6745\frac{x-\text{median}}{\text{MAD}}$, flag $|M| > 3.5$ (robust).

```
f = df["fare"]
z = (f - f.mean()) / f.std(); f[z.abs() > 3]
q1, q3 = f.quantile([.25, .75]); iqr = q3 - q1
lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
out = f[(f < lo) | (f > hi)]
# treatment
f.clip(lo, hi)                                    # cap
from scipy.stats.mstats import winsorize
winsorize(f.to_numpy(), limits=[.05, .05])        # 5% each tail
df[(f >= lo) & (f <= hi)]                         # trim rows
np.log1p(f)                                       # transform
```

LOF vs DBSCAN vs Isolation Forest

	LOF	DBSCAN	Isolation Forest
idea	local density vs its k neighbours'	clusters dense regions; points in no cluster are noise	random splits isolate outliers in few cuts
score	LOF ≈ 1 normal, $\gg 1$ outlier	label -1 = noise	short average path = outlier
params	`n_neighbors`, contamination	`eps`, `min_samples`	`n_estimators`, contamination
good at	varying density, local outliers	arbitrary shapes, also clusters	high dimensions, large n, fast
weak at	large n (slow), choosing k	choosing eps; varying density	local outliers in dense areas

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import LocalOutlierFactor
from sklearn.cluster import DBSCAN
from sklearn.ensemble import IsolationForest
X = StandardScaler().fit_transform(df[["age","fare"]].dropna())  #
      ↪ scale!
lof = LocalOutlierFactor(n_neighbors=20, contamination=.05).
      ↪ fit_predict(X)
db  = DBSCAN(eps=.5, min_samples=5).fit_predict(X)
iso = IsolationForest(contamination=.05, random_state=42).
      ↪ fit_predict(X)
(lof == -1).sum(), (db == -1).sum(), (iso == -1).sum()   # 36, 22,
      ↪ 36 of 714
```

Trap

Check feasibility before deleting (age 200 is an error; fare 512 is real). The z-rule is masked by the outlier itself; IQR and modified z are robust. Scale features before LOF / DBSCAN (distance based).

Variations

```
def iqr_count(x):
    q1, q3 = x.quantile([.25, .75]); i = q3 - q1
    return ((x < q1 - 1.5*i) | (x > q3 + 1.5*i)).sum()
num.apply(iqr_count)    # age 11, sibsp 46, parch 213, fare 116
zs = num.apply(lambda c: stats.zscore(c, nan_policy="omit"))
(zs.abs() > 3).sum()    # age 2, sibsp 30, parch 15, fare 20
f2 = f.copy(); f2[f2 > hi] = f.median()     # replace with median
df["fare_outlier"] = (f > hi).astype(int)   # flag instead of drop
# multivariate: Mahalanobis distance
X = df[["age","fare"]].dropna().to_numpy()
mu = X.mean(0); VI = np.linalg.inv(np.cov(X, rowvar=False))
md2 = np.einsum("ij,jk,ik->i", X - mu, VI, X - mu)
(md2 > stats.chi2.ppf(.975, df=2)).sum()      # 29 of 714
from sklearn.covariance import EllipticEnvelope
EllipticEnvelope(contamination=.05, random_state=0).fit_predict(X)
```

11. Transformation: scaling, function, power

Four normalization / scaling methods

Min–max: $x' = \frac{x-x_{\min}}{x_{\max}-x_{\min}} \in [0,1]$ (sensitive to outliers).
Z-score (standardization): $z = \frac{x-\mu}{\sigma}$, mean 0, SD 1.
Robust: $x' = \frac{x-\text{median}}{\text{IQR}}$ (resists outliers).
Max-abs: $x' = \frac{x}{\max|x|} \in [-1,1]$ (keeps zeros, sparse data).
Also: decimal scaling $x/10^k$; `Normalizer` scales each *row* to length 1.

```
from sklearn.preprocessing import (MinMaxScaler, StandardScaler,
    RobustScaler, MaxAbsScaler, Normalizer)
v = df[["fare"]]                  # 2-D (DataFrame) for sklearn
# 1. min-max      by hand, then sklearn
(v - v.min()) / (v.max() - v.min())
MinMaxScaler().fit_transform(v)
# 2. z-score
(v - v.mean()) / v.std(ddof=0)
StandardScaler().fit_transform(v)
# 3. robust
(v - v.median()) / (v.quantile(.75) - v.quantile(.25))
RobustScaler().fit_transform(v)
# 4. max-abs
v / v.abs().max()
MaxAbsScaler().fit_transform(v)
# row-wise unit length
Normalizer(norm="l2").fit_transform(df[["age","fare"]].dropna())
```

Scaling changes units, *not* shape: skew is unchanged. To change shape, use a function or power transform.

Function transformers (fixed formula)

- **Log** $\ln(1+x)$: strong right skew, multiplicative data (income, fare).
- **Square root** $\sqrt{x}$: moderate right skew, counts.
- **Reciprocal** $1/x$: very strong right skew; reverses order.
- **Square** x^2 / **exponential** e^x: *left* skew.

```
from sklearn.preprocessing import FunctionTransformer
FunctionTransformer(np.log1p).fit_transform(v)       # log(1+x)
FunctionTransformer(np.sqrt).fit_transform(v)
FunctionTransformer(lambda t: 1/(t + 1)).fit_transform(v)
FunctionTransformer(np.square).fit_transform(v)
```

Power transformers (learn λ from the data)

Box–Cox (data > 0): $y = \frac{x^\lambda - 1}{\lambda}$ ($\lambda \neq 0$), $\ln x$ ($\lambda = 0$). Yeo–Johnson: same idea, also handles 0 and negatives. λ chosen by maximum likelihood to make the data as normal as possible; `PowerTransformer` also standardizes.

```
from sklearn.preprocessing import PowerTransformer,
      ↪ QuantileTransformer
PowerTransformer(method="box-cox").fit_transform(v + 1)    # needs
      ↪ > 0
PowerTransformer(method="yeo-johnson").fit_transform(v)    # any
      ↪ real
stats.boxcox(f + 1)                              # (values, best lambda)
QuantileTransformer(n_quantiles=100,
    output_distribution="normal").fit_transform(v)   # forces a
      ↪ shape
```

Trap

Fit scalers and transformers on the training set only, then `transform` the test set. sklearn wants 2-D input: `df[["fare"]]`, not `df["fare"]`.

Variations

```
numc = df[["age","fare","sibsp"]].dropna()
sc = StandardScaler().set_output(transform="pandas")  # keeps names
scaled = sc.fit_transform(numc); sc.inverse_transform(scaled)
pd.DataFrame(MinMaxScaler(feature_range=(-1, 1)).fit_transform(numc
      ↪ ),
             columns=numc.columns)
tr, te = train_test_split(numc, test_size=.2, random_state=0)
sc2 = StandardScaler().fit(tr); sc2.transform(te)  # no leakage
QuantileTransformer(n_quantiles=100,
    output_distribution="uniform").fit_transform(numc[["fare"]])
from sklearn.preprocessing import Binarizer
Binarizer(threshold=30).fit_transform(numc[["fare"]])   # 0/1
np.log10(f + 1); np.log2(f + 1); np.cbrt(f)   # cbrt allows
      ↪ negatives
pt = PowerTransformer().fit(numc[["fare"]]); pt.lambdas_  # -0.15
pd.DataFrame({"raw": num.skew(),
              "log1p": np.log1p(num.clip(lower=0)).skew()})
```

Do not transform blindly: log1p takes fare's skew $4.79 \to 0.39$, but age $0.39 \to -1.88$ (now left skewed). Check the skew after every transform.

12. Binning and encoding

```
pd.cut(df["age"], bins=4)                          # equal width
pd.qcut(df["fare"], q=4, labels=["Q1","Q2","Q3","Q4"])   # equal
    ↪ count
pd.cut(df["age"], bins=[0, 12, 19, 59, 100],
       labels=["child", "teen", "adult", "senior"])     # domain
from sklearn.preprocessing import KBinsDiscretizer
KBinsDiscretizer(n_bins=4, encode="ordinal",
                 strategy="kmeans").fit_transform(df[["fare"]])  #
    ↪ cluster
```

Which encoder

nominal, few levels	one-hot (drop one with an intercept)
ordinal	ordinal encoder with the order given
target labels y	LabelEncoder
many levels	binary, frequency, target (mean), hashing

```
pd.get_dummies(df["embarked"], prefix="emb", drop_first=True)
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder,
    ↪ LabelEncoder
OneHotEncoder(drop="first", handle_unknown="ignore") \
    .fit_transform(df[["embarked"]].astype(object).fillna("NA"))
OrdinalEncoder(categories=[["Third", "Second", "First"]]) \
    .fit_transform(df[["class"]].astype(object))
LabelEncoder().fit_transform(df["sex"])
df["embarked"].map(df["embarked"].value_counts())            #
    ↪ frequency
df["embarked"].map(df.groupby("embarked")["survived"].mean())  #
    ↪ target
```

Trap

Dummy-variable trap: all k dummies sum to 1, so drop one. Target encoding leaks y: compute on training folds only. LabelEncoder on features invents an order.

Variations

```
pd.cut(df["age"], bins=[0,18,60,100], right=False, include_lowest=
    ↪ True)
cats, edges = pd.cut(df["age"], 4, retbins=True)   # get the edges
pd.qcut(df["fare"], 10, duplicates="drop").value_counts()
pd.get_dummies(df, columns=["sex","embarked"], drop_first=True,
               dtype=int).shape                    # (891, 16)
oh = OneHotEncoder(sparse_output=False, handle_unknown="ignore") \
       .fit(df[["sex"]].astype(object))
oh.get_feature_names_out()          # ['sex_female', 'sex_male']
df["class"].map({"Third": 1, "Second": 2, "First": 3})  # ordinal
    ↪ by hand
df["class"].cat.codes
```

Fare quartile edges: 0, 7.91, 14.45, 31.00, 512.33. Age into child (<18) / adult / senior (≥ 60): 139 / 553 / 22.

13. Numeric and categorical together (sklearn)

```
from sklearn.compose import ColumnTransformer, make_column_selector
    ↪ as sel
from sklearn.pipeline import Pipeline
pre = ColumnTransformer([
  ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                    ("sc", StandardScaler())]), sel(dtype_include="
    ↪ number")),
  ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")
    ↪ ),
                    ("oh", OneHotEncoder(handle_unknown="ignore"))
    ↪ ]),
   ["sex", "embarked"])])
X = df[["age","fare","sibsp","sex","embarked"]].astype(
       {"sex": object, "embarked": object})
pre.fit_transform(X).shape                  # (891, 8)
```

14. Cleaning checklist

```
df.columns = df.columns.str.strip().str.lower()     # names
df["name"] = df["name"].str.strip().str.title()     # text values
df["marks"] = pd.to_numeric(df["marks"], errors="coerce")  # junk
    ↪ -> NaN
df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y")
df["roll"] = df["roll"].astype(str)                 # int64 ->
    ↪ string
df["city"] = df["city"].astype("category")
df["marks"].between(0, 100).all()                   # feasible
    ↪ range
df.duplicated().sum(); df.drop_duplicates(subset=["name"])
```

Order: features (dimension, names, types, units, dates, duplicates) → values (range, feasibility, missing, outliers) → transform → save.

Variations

```
d.columns = [c.strip().lower() for c in d.columns]
d["price"] = d["price"].str.replace(r"[$,\s]", "", regex=True).
    ↪ astype(float)
d["city"] = d["city"].str.strip().str.title()  # 'delhi ','DELHI' ->
    ↪  'Delhi'
d = d.loc[:, d.nunique() > 1]                  # drop constant
    ↪ columns
obj = d.select_dtypes(exclude="number").columns
d[obj] = d[obj].apply(lambda c: c.str.strip()) # strip every text
    ↪ column
d.drop_duplicates().reset_index(drop=True)
```

Titanic (seaborn version) has 107 duplicate rows: it has no name or ticket column, so different passengers look identical. Check before you drop.

A. Write N, O, I or R

1	Blood group	N
2	Aadhaar / phone number	N
3	PIN code	N
4	Hotel star rating	O
5	Education level (school, UG, PG)	O
6	Calendar year	I
7	Temperature in °C	I
8	Temperature in kelvin	R
9	IQ score	I
10	Credit score (300–900)	I
11	Weight (kg)	R
12	Monthly salary	R
13	Number of children	R
14	Time to finish a race (s)	R
15	Clock time of arrival	I
16	Date of birth	I
17	Age in years	R
18	Movie genre	N
19	T-shirt size S/M/L/XL	O
20	Pain scale 0–10	O
21	Finishing position in a marathon	O
22	Latitude	I
23	Distance from home (km)	R
24	Electricity used (kWh)	R
25	Agree / neutral / disagree	O
26	Number of website visits	R
27	Jersey number	N
28	Product code SKU-482	N
29	Income bracket (low, mid, high)	O
30	Height of a building (m)	R

Tests: *Can I order it?* (no: N). *Are the steps equal?* (no: O). *Does 0 mean "none"?* (no: I, yes: R). A number used as a label (PIN, jersey) is still N.

B. True or false (with the reason)

1. Median is a valid average for ordinal data. **T**: needs only order.
2. Mean of nominal codes (1 = red, 2 = blue) is meaningful. **F**: codes are labels.
3. 20°C is twice as hot as 10°C. **F**: interval, no true zero.
4. 40 kg is twice 20 kg. **T**: ratio scale.
5. A box plot shows the five-number summary. **T**.
6. A histogram is suitable for nominal data. **F**: use a bar chart.
7. Pie charts suit many categories. **F**: keep to a few.
8. Correlation $r = 0$ means no relationship. **F**: only no *linear* one.
9. Spearman suits ordinal data. **T**: works on ranks.
10. Pearson r is resistant to outliers. **F**: one point can flip it.
11. Right skew: mean > median. **T**.
12. Skewness 0 proves symmetry. **F**: a lopsided set can have 0.
13. Kurtosis > 0 (excess) means heavy tails. **T**.
14. MCAR can be tested from the data. **T**: Little's test.
15. MAR and MNAR can be told apart from the data alone. **F**.
16. Listwise deletion is unbiased under MAR. **F**: only under MCAR.
17. Mean imputation keeps the variance. **F**: it shrinks it.
18. missingno heatmap shows correlation of values. **F**: of missingness.
19. The IQR rule assumes normal data. **F**: the z-rule does.
20. Every outlier should be deleted. **F**: check feasibility first.
21. Min–max scaling changes the shape of a distribution. **F**: only its range.
22. Standardization gives mean 0, SD 1. **T**.
23. Robust scaling uses the median and IQR. **T**.
24. Box–Cox works on negative values. **F**: Yeo–Johnson does.
25. Log transform reduces right skew. **T**.
26. Square transform reduces left skew. **T**.
27. One-hot encoding suits ordinal data best. **F**: ordinal encoding keeps order.
28. Label encoding a nominal feature can mislead a linear model. **T**: invents an order.
29. Equal-frequency bins have equal widths. **F**: equal counts.
30. Scalers should be fit on the whole dataset before splitting. **F**: train only (leakage).

C. One-line code (df = titanic)

```python
# 1  shape, number of rows only
df.shape; len(df)
# 2  data type of each column
df.dtypes
# 3  missing values per column, and in %
df.isnull().sum(); df.isnull().mean()*100
# 4  total missing cells
df.isnull().sum().sum()
# 5  rows with no missing value
df.dropna()                                  # 182 rows
# 6  drop a column with > 50% missing
df.drop(columns=df.columns[df.isna().mean() > .5])
# 7  fill age with median, embarked with mode
df["age"].fillna(df["age"].median())
df["embarked"].fillna(df["embarked"].mode()[0])
# 8  numeric columns only
df.select_dtypes(include="number")
# 9  categorical columns only
df.select_dtypes(exclude="number")
# 10 mean of every numeric column
df.mean(numeric_only=True)
# 11 five-number summary of fare
df["fare"].quantile([0, .25, .5, .75, 1])
# 12 skewness of all numeric columns
df.skew(numeric_only=True)
# 13 count of each class
df["class"].value_counts()
# 14 percentage of each class
df["class"].value_counts(normalize=True)*100
# 15 survival rate by sex
df.groupby("sex")["survived"].mean()   # F .742, M .189
# 16 mean fare by class and sex
df.pivot_table("fare", "class", "sex", observed=True)
# 17 passengers older than 60
df[df["age"] > 60]
# 18 females in first class
df[(df["sex"]=="female") & (df["pclass"]==1)]
# 19 10 most expensive tickets
df.nlargest(10, "fare")
# 20 sort by age descending
df.sort_values("age", ascending=False)
# 21 number of unique values per column
df.nunique()
# 22 count duplicate rows
df.duplicated().sum()                        # 107
# 23 rename a column
df.rename(columns={"sex": "gender"})
# 24 new column family size
df["family"] = df["sibsp"] + df["parch"] + 1
# 25 convert int64 to string
df["pclass"].astype(str)
# 26 text to number, junk to NaN
pd.to_numeric(s, errors="coerce")
# 27 correlation matrix
df.corr(numeric_only=True)
# 28 Spearman between age and fare
df["age"].corr(df["fare"], method="spearman")
# 29 crosstab sex vs survived
pd.crosstab(df["sex"], df["survived"])
# 30 IQR of fare
df["fare"].quantile(.75) - df["fare"].quantile(.25)
# 31 cap fare at the 99th percentile
df["fare"].clip(upper=df["fare"].quantile(.99))
# 32 z-scores of age
stats.zscore(df["age"], nan_policy="omit")
# 33 min-max scale fare by hand
(df["fare"]-df["fare"].min())/(df["fare"].max()-df["fare"].min())
# 34 log transform fare (zeros present)
np.log1p(df["fare"])
# 35 bin age into 4 equal-width bins
pd.cut(df["age"], 4)
# 36 fare into quartiles
pd.qcut(df["fare"], 4)
```

```
# 37 one-hot sex and embarked
pd.get_dummies(df, columns=["sex","embarked"], drop_first=True)
# 38 random 10% sample, fixed seed
df.sample(frac=.1, random_state=42)
# 39 histogram of every numeric column
df.select_dtypes("number").hist(figsize=(10,6))
# 40 save without the index
df.to_csv("titanic_clean.csv", index=False)
```

D. Theory + code (model-paper style)

D1. What is kurtosis? Types? Print it for titanic age and fare. Peakedness/tail weight relative to a normal. Excess kurtosis = $\frac{\frac{1}{n}\sum(x-\bar{x})^4}{s^4} - 3$. Mesokurtic ($\approx 0$, normal), leptokurtic ($> 0$, sharp peak, heavy tails, more outliers), platykurtic (< 0, flat, light tails, e.g. uniform).

```
df[["age","fare"]].kurt()      # age 0.18 (meso), fare 33.4 (lepto)
```

D2. Explain the IQR rule. Count outliers in every numeric column. IQR $= Q_3 - Q_1$; fences $Q_1 - 1.5\,\text{IQR}$, $Q_3 + 1.5\,\text{IQR}$; beyond them is an outlier ($3\,\text{IQR}$: extreme). Uses quartiles, so robust to skew and to the outliers themselves.

```
def iqr_count(x):
    q1, q3 = x.quantile([.25,.75]); i = q3 - q1
    return ((x < q1-1.5*i) | (x > q3+1.5*i)).sum()
df.select_dtypes("number").apply(iqr_count)
# age 11, sibsp 46, parch 213, fare 116
```

D3. Compare z-score, IQR and modified z for fare. z: $|z| > 3$, assumes normality, masked by outliers (20 flagged). IQR: no distribution assumption (116). Modified z: median and MAD, $|M| > 3.5$, most robust (160, because fare is so skewed). Pick IQR/modified z for skewed data.

D4. Techniques for missing values; apply to titanic. Deletion: listwise (`dropna()`, 891→182 rows: wasteful), pairwise, drop column (deck 77% missing). Imputation: mean/median/mode, group-wise, ffill/bfill/interpolate (ordered data), KNN, regression/iterative (MICE), add an indicator. Model-based: maximum likelihood/EM.

```
df = df.drop(columns="deck")
df["age"] = df["age"].fillna(df.groupby(["pclass","sex"])["age"]
                             .transform("median"))
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
df.isnull().sum().sum()        # only embark_town left
```

D5. How do you check whether age is MCAR? Compare rows with and without age: if missingness is MCAR, other columns should look the same.

```
miss = df["age"].isna()
pd.crosstab(miss, df["pclass"], normalize="columns")
stats.chi2_contingency(pd.crosstab(miss, df["pclass"]))[1]  # 1e-10
```

Missing in 27.7% of 3rd class vs 6% of 2nd: not MCAR (MAR on class). Formal test: Little's MCAR (χ^2, H_0: MCAR). msno heatmap/dendrogram show it visually.

D6. Normalization vs standardization vs robust scaling: when? Min–max to $[0, 1]$ when a bounded range is needed (images, distance methods) and there are no big outliers. Standardization when features have different units and roughly normal shape (PCA, linear models, SVM). Robust when outliers are present. None changes the shape. Code: §11.

D7. Box–Cox vs Yeo–Johnson. Find λ for fare. Both search λ by maximum likelihood to make data closest to normal. Box–Cox needs $x > 0$ ($\lambda = 0$ is log, $\frac{1}{2}$ square root, -1 reciprocal); Yeo–Johnson handles 0 and negatives.

```
stats.boxcox(df["fare"] + 1)[1]          # -0.098
PowerTransformer().fit(df[["fare"]]).lambdas_   # yeo-johnson -0.15
```

$\lambda \approx 0$: close to a log transform. Skew $4.79 \to -0.04$.

D8. Equal-width vs equal-frequency binning on fare.

```
pd.cut(df["fare"], 4).value_counts().sort_index()   # 853, 29, 6, 3
pd.qcut(df["fare"], 4).value_counts().sort_index()  # 223, 224,
    ↪ 222, 222
```

Equal width on skewed data dumps nearly everything in one bin; equal frequency balances counts (edges 7.91, 14.45, 31). Domain bins carry meaning; k-means bins follow clusters.

D9. Encoding techniques, with titanic examples. One-hot: sex, embarked (`get_dummies(drop_first=True)`, dummy trap). Ordinal: class, with the order given (Third < Second < First) to `OrdinalEncoder`. Label: the target only. Frequency: embarked → 644/168/77. Target (mean): embarked → survival rate (fit on train only). Binary: $\lceil \log_2 k \rceil$ columns. Hashing: very many levels.

D10. Correlation types; heatmap for titanic; interpret. Pearson (linear, numeric), Spearman / Kendall (monotone, ranks, ordinal, outliers), point-biserial (binary vs numeric), Cramér's V (two categoricals).

```
sns.heatmap(df.corr(numeric_only=True), annot=True, fmt=".2f",
            cmap="coolwarm", vmin=-1, vmax=1)
```

fare–pclass -0.55: higher class number, cheaper ticket. Spearman -0.69 is stronger because fare is skewed. Correlation is not causation.

D11. Univariate vs bivariate vs multivariate, with a plot each. Univariate: one variable's distribution (`histplot(age)`, `countplot(class)`). Bivariate: two at once (`boxplot(x="class", y="fare")`, `crosstab(sex, survived)`, scatter age–fare). Multivariate: three or more (`pairplot(hue="survived")`, correlation heatmap, `catplot(col=)`).

D12. Sampling methods; code on titanic.

```
df.sample(n=90, random_state=1)                       # SRS
df.groupby("class", observed=True, group_keys=False) \
  .sample(frac=.1, random_state=1)    # stratified: 49, 22, 18
df.iloc[::10]                         # systematic, every 10th
```

Stratified keeps the class proportions; cluster sampling picks whole groups (e.g. all passengers of two decks).

D13. Write a data-quality report function.

```
def quality_report(data):
    print("shape:", data.shape, "dupes:", data.duplicated().sum())
    rep = pd.DataFrame({"dtype": data.dtypes.astype(str),
        "missing": data.isna().sum(),
        "missing_%": (data.isna().mean()*100).round(1),
        "unique": data.nunique()})
    print(rep.sort_values("missing_%", ascending=False))
    print(data.describe().T[["mean","std","min","50%","max"]])
    return rep
quality_report(df)   # deck 77.2%, age 19.9%, embarked 0.2%
```

D14. Explain each row of `describe()`. count (non-missing), mean, std (sample, $n-1$), min, 25% (Q_1), 50% (median), 75% (Q_3), max. `describe(include="object")`: count, unique, top (mode), freq (mode's count). The gap between mean and 50% hints at skew (fare 32.20 vs 14.45).

D15. Preprocess numeric and categorical columns together. `ColumnTransformer`: numeric → median imputer + `StandardScaler`; categorical → mode imputer + `OneHotEncoder(handle_unknown="ignore")`; wrap in a `Pipeline`; `fit` on train, `transform` test. Code: §13.

Exam habits

Write imports once at the top. Name the dataset (`df = sns.load_dataset("titanic")`). After the code, write one line saying what the output means. For sklearn pass 2-D input (`df[["col"]]`).